

**FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA -  
FIAP**

**INTELIGÊNCIA ARTIFICIAL**

# **EDGE COMPUTING E RESILIÊNCIA OFFLINE NO PROJETO CARDIOIA**

**ACADÊMICOS:**

VINÍCIUS PEREIRA SANTANA

**RM:** 564940

VITOR AUGUSTO PRADO GUISSO

**RM:** 562317

**TURMA:**

1 TIAOA-2026

**12 DE MAIO DE 2026**

## SUMÁRIO

|                                                 |    |
|-------------------------------------------------|----|
| 1 INTRODUÇÃO .....                              | 3  |
| 1.1 Objetivo da solução .....                   | 3  |
| 2 FUNDAMENTAÇÃO TEÓRICA .....                   | 4  |
| 2.1 Internet das Coisas (IoT) .....             | 4  |
| 2.2 Edge Computing .....                        | 4  |
| 2.3 MQTT .....                                  | 5  |
| 3 ARQUITETURA DO SISTEMA .....                  | 5  |
| 3.1 Sensores utilizados .....                   | 5  |
| 4 PROCESSAMENTO LOCAL (EDGE COMPUTING) .....    | 6  |
| 4.1 Simulação de batimentos cardíacos .....     | 6  |
| 5 RESILIÊNCIA OFFLINE .....                     | 7  |
| 5.1 Quando online .....                         | 7  |
| 5.2 Quando offline .....                        | 7  |
| 5.3 Quando a conexão retorna .....              | 7  |
| 5.4 Lógica de funcionamento da resiliência..... | 8  |
| 6 FLUXO DE FUNCIONAMENTO .....                  | 8  |
| 6.1 Fluxo de comunicação MQTT e dashboard.....  | 9  |
| 7 SIMULAÇÃO DE CENÁRIOS CLÍNICOS .....          | 10 |
| 8 RESULTADOS .....                              | 11 |
| 9 LIMITAÇÕES DA SOLUÇÃO .....                   | 13 |
| 10 TRABALHOS FUTUROS .....                      | 13 |
| 11 CONCLUSÃO .....                              | 13 |
| 12 REFERÊNCIAS .....                            | 14 |

## 1 INTRODUÇÃO

O projeto CardioIA foi desenvolvido com o objetivo de simular um sistema inteligente de monitoramento cardíaco utilizando conceitos de Internet das Coisas (IoT), Edge Computing e comunicação em nuvem. A proposta busca demonstrar, de forma prática, como dispositivos embarcados podem ser utilizados na área da saúde para captura, processamento e transmissão de sinais vitais em tempo real.

O sistema foi implementado utilizando um ESP32 no ambiente de simulação Wokwi, juntamente com sensores simulados capazes de representar sinais fisiológicos de um paciente. Além da coleta dos dados, o projeto também implementa mecanismos de resiliência offline, permitindo que o sistema continue funcionando mesmo em situações de perda de conexão.

O projeto foi desenvolvido utilizando ESP32, protocolo MQTT, broker público EMQX e dashboard Node-RED, permitindo a construção de um fluxo completo de IoT aplicado à saúde digital.

### 1.1 Objetivo da solução

A solução desenvolvida tem como objetivo monitorar sinais vitais simulados de um paciente cardiológico, realizando:

- Captura de temperatura corporal;
- Captura de umidade;
- Simulação de batimentos cardíacos;
- Processamento local dos dados;
- Detecção automática de alertas clínicos;
- Transmissão via MQTT;
- Armazenamento temporário local em situações offline;
- Sincronização automática após reconexão.

O sistema busca demonstrar conceitos modernos de Edge Computing aplicados à saúde digital, especialmente em ambientes críticos onde falhas de conectividade não podem interromper completamente o monitoramento.

## **2 FUNDAMENTAÇÃO TEÓRICA**

### **2.1 Internet das Coisas (IoT)**

A Internet das Coisas (IoT) representa a conexão entre dispositivos físicos capazes de coletar, processar e transmitir dados através da internet. Esses dispositivos utilizam sensores, atuadores e sistemas embarcados para gerar informações em tempo real.

Na área da saúde, a IoT possibilita monitoramento remoto de pacientes, acompanhamento contínuo de sinais vitais e automação de processos clínicos, permitindo respostas mais rápidas em situações de emergência.

O projeto CardioIA aplica esse conceito utilizando sensores conectados a um ESP32, responsável pela coleta e transmissão dos dados clínicos simulados.

### **2.2 Edge Computing**

Edge Computing é um paradigma computacional no qual parte do processamento ocorre diretamente no dispositivo de borda, reduzindo a dependência da nuvem.

Ao invés de enviar dados brutos continuamente para servidores remotos, o dispositivo realiza parte da inteligência localmente, trazendo vantagens como:

- Redução de latência;
- Menor consumo de banda;
- Respostas mais rápidas;
- Maior resiliência em falhas de conexão.

No CardioIA, o ESP32 realiza processamento local dos sinais vitais, detectando possíveis estados clínicos antes mesmo da transmissão via MQTT.

### **2.3 MQTT**

O MQTT (Message Queuing Telemetry Transport) é um protocolo leve de comunicação baseado no modelo publish/subscribe.

Ele é amplamente utilizado em aplicações IoT devido ao:

- Baixo consumo de rede;
- Alta eficiência;
- Simplicidade de implementação;
- Capacidade de funcionar em dispositivos limitados.

No projeto, foi utilizado o broker público EMQX ([broker.emqx.io](https://broker.emqx.io)) para transmissão dos dados entre o ESP32 e o dashboard Node-RED.

### 3 ARQUITETURA DO SISTEMA

O sistema foi dividido em seis etapas principais:

- Sensores
- ESP32
- Processamento local
- MQTT broker
- Node-RED
- Dashboard

O ESP32 realiza a leitura dos sensores e executa a lógica de processamento local. Após isso, os dados são transmitidos via MQTT para o broker EMQX.

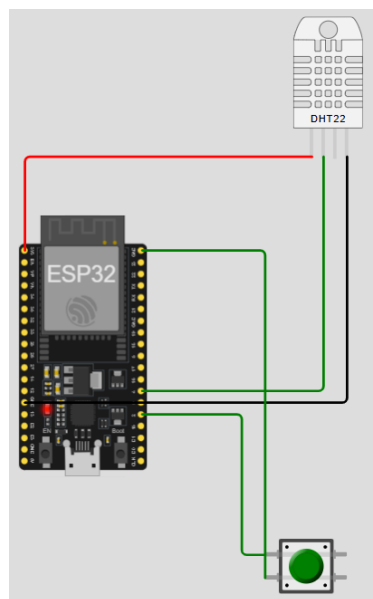
O Node-RED recebe essas informações e atualiza o dashboard em tempo real.

#### 3.1 Sensores utilizados

O projeto utiliza dois sensores distintos:

- Sensor DHT22: simulação de temperatura e umidade.
- Botão de pressão: simulação de batimentos cardíacos.

O botão foi utilizado para representar eventos de pulsação, permitindo gerar diferentes cenários clínicos durante a simulação. A figura 1 mostra a configuração no Wokwi, em que foi conectado no ESP32 um DHT22 e um botão.



**Figura 1 – Configuração Wokwi**

## 4 PROCESSAMENTO LOCAL (EDGE COMPUTING)

O ESP32 realiza o processamento dos dados localmente antes do envio para a nuvem. Essa abordagem caracteriza o conceito de Edge Computing, onde parte da inteligência do sistema ocorre diretamente no dispositivo de borda.

Durante a execução, o ESP32:

- Lê os sensores;
- Calcula os batimentos cardíacos simulados;
- Classifica possíveis estados clínicos;
- Decide se os dados serão enviados ou armazenados localmente.

As regras implementadas incluem:

- Hipotermia: temperatura abaixo de 35 °C.
- Febre: temperatura igual ou superior a 38 °C.
- Bradicardia: BPM abaixo de 50.
- Taquicardia: BPM acima de 120.
- Ausência de batimentos: BPM igual a 0.

Além disso, o sistema também detecta situações críticas combinadas, como:

- Febre + taquicardia;
- Hipotermia + bradicardia;
- Hipotermia + ausência de batimentos.

### 4.1 Simulação de batimentos cardíacos

Para simular os batimentos cardíacos no ambiente Wokwi, foi utilizado um botão conectado ao ESP32.

Cada acionamento do botão representa um pulso cardíaco. O sistema contabiliza a quantidade de pulsos detectados dentro de uma janela de tempo pré-definida.

Com base nessa quantidade de pulsos, o ESP32 calcula o BPM (Batimentos Por Minuto) utilizando uma lógica proporcional.

O funcionamento ocorre da seguinte forma:

- Cada clique no botão representa um batimento;
- O ESP32 contabiliza os pulsos recebidos;
- Após um intervalo de tempo, o sistema calcula o BPM equivalente;
- O valor é então utilizado na análise clínica local.

Essa abordagem permitiu simular diferentes cenários fisiológicos, como:

- Paciente normal;
- Bradicardia;
- Taquicardia;
- Ausência de batimentos.

Além disso, o sistema implementa detecção de ausência de sinal cardíaco. Caso nenhum pulso seja identificado durante o intervalo de leitura, o BPM é definido como zero, permitindo gerar alertas críticos relacionados à ausência de batimentos.

Toda essa lógica é executada localmente no ESP32 antes do envio das informações via MQTT, reforçando o conceito de Edge Computing aplicado no projeto.

## **5 RESILIÊNCIA OFFLINE**

Um dos principais objetivos desta etapa foi implementar resiliência offline.

Como o simulador Wokwi possui limitações relacionadas ao SPIFFS, foi adotada uma estratégia alternativa utilizando memória local temporária (cache em RAM), conforme permitido no enunciado.

O sistema simula automaticamente a perda de conectividade Wi-Fi por meio de uma variável booleana.

O funcionamento ocorre da seguinte forma:

### **5.1 Quando online**

- O ESP32 envia os dados normalmente via MQTT;
- Os dados chegam em tempo real ao dashboard.

### **5.2 Quando offline**

- O sistema continua realizando leituras normalmente;
- Os dados são armazenados localmente em um vetor de cache;
- Nenhuma informação é perdida durante a desconexão.

### **5.3 Quando a conexão retorna**

- O ESP32 identifica a reconexão;
- Todos os dados armazenados são sincronizados automaticamente;
- O cache é limpo após o envio bem-sucedido.

Essa lógica demonstra um comportamento importante em aplicações médicas críticas, onde a perda de conectividade não pode interromper completamente o monitoramento.

#### 5.4 Lógica de funcionamento da resiliência

O mecanismo de resiliência offline foi implementado utilizando uma estratégia baseada em cache temporário em memória RAM no ESP32.

Durante a execução do sistema, uma variável booleana controla o estado da conectividade simulada do dispositivo.

Quando o sistema identifica que a conexão está ativa, os dados coletados são imediatamente serializados em formato JSON e enviados ao broker MQTT.

Entretanto, quando a conexão é interrompida, o ESP32 altera automaticamente seu comportamento operacional. Nesse estado, os dados deixam de ser transmitidos e passam a ser armazenados localmente em um vetor de cache temporário.

Cada leitura armazenada contém:

- Temperatura;
- Umidade;
- Batimentos cardíacos;
- Timestamp da leitura;
- Estado clínico identificado.

Durante o período offline, o sistema continua executando normalmente:

- Leitura dos sensores;
- Classificação clínica;
- Processamento local;
- Geração de alertas.

Após o retorno da conectividade, o ESP32 executa automaticamente uma rotina de sincronização.

Essa rotina percorre todo o vetor de cache, enviando individualmente cada leitura armazenada anteriormente. Após confirmação do envio, o cache é limpo automaticamente.

Essa estratégia garante continuidade operacional mesmo em cenários de falha de rede, característica importante em aplicações médicas críticas e sistemas IoT resilientes.

## 6 FLUXO DE FUNCIONAMENTO

O fluxo geral da aplicação ocorre conforme as etapas abaixo:

1. O ESP32 realiza a leitura dos sensores;
2. Os dados são processados localmente;
3. O sistema verifica o estado da conectividade;
4. Caso esteja online: os dados são enviados via MQTT;



5. Caso esteja offline: os dados são armazenados localmente;
6. Após reconexão: os dados pendentes são sincronizados;
7. O Node-RED recebe os dados e atualiza o dashboard.

### 6.1 Fluxo de comunicação MQTT e dashboard

A comunicação entre o ESP32 e o dashboard foi implementada utilizando o protocolo MQTT no modelo publish/subscribe.

O ESP32 atua como publisher, sendo responsável pela publicação dos dados clínicos simulados.

O broker público EMQX foi utilizado como intermediador da comunicação, recebendo as mensagens publicadas pelo ESP32 e distribuindo-as aos dispositivos inscritos no tópico correspondente.

O tópico MQTT utilizado foi: cardioia/sensores

As mensagens foram transmitidas em formato JSON contendo:

- Temperatura;
- Umidade;
- bpm;
- Timestamp;
- Status clínico;
- Indicadores booleanos de alerta.

*Exemplo de payload transmitido:*

```
{
  "temperatura": 40.5,
  "umidade": 40,
  "bpm": 125,
  "febre": true,
  "taquicardia": true,
  "status_paciente": "febre_taquicardia"
}
```

No Node-RED, foi criado um fluxo responsável por:

- Receber os dados MQTT;
- Interpretar o JSON;
- Processar os estados clínicos;
- Atualizar o dashboard em tempo real.

O dashboard foi composto pelos seguintes elementos:

- Gráfico temporal de BPM;

- Gauge de temperatura;
- Painel textual de status do paciente;
- Alertas visuais coloridos.

As cores do dashboard variam dinamicamente conforme o estado clínico detectado:

- Verde para paciente normal;
- Amarelo para alertas moderados;
- Vermelho para alertas críticos.

Essa arquitetura permitiu desacoplamento entre captura de dados, transmissão e visualização, característica comum em sistemas modernos de IoT distribuída.

## **7 SIMULAÇÃO DE CENÁRIOS CLÍNICOS**

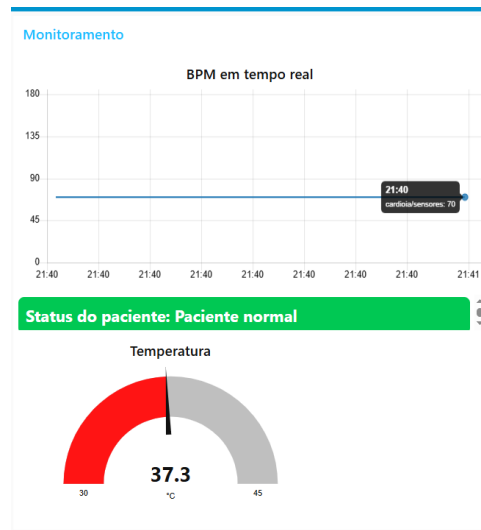
Durante os testes, foram simulados diversos estados fisiológicos:

- Paciente normal;
- Febre;
- Hipotermia;
- Bradicardia;
- Taquicardia;
- Ausência de batimentos;
- Alertas críticos combinados.

Esses cenários permitiram validar o funcionamento do sistema e demonstrar a capacidade do ESP32 em processar dados clínicos localmente.

## 8 RESULTADOS

O sistema apresentou funcionamento estável durante a transmissão online, exibindo os dados em tempo real no dashboard do Node-RED, como mostra a figura 2.



**Figura 2 – Dashboard de monitoramento em tempo real no Node-RED.**

A Figura 2 mostra a simulação de um monitoramento em tempo real em que o paciente está com batimentos em 70 bpm e temperatura em 37,3 °C. Portanto, se enquadra com o status de normal.

Durante a perda de conectividade, o sistema continuou coletando os dados normalmente e armazenando localmente no cache do ESP32. Após o retorno da conectividade, os dados armazenados localmente foram sincronizados automaticamente via MQTT. Como mostra a Figura 3.

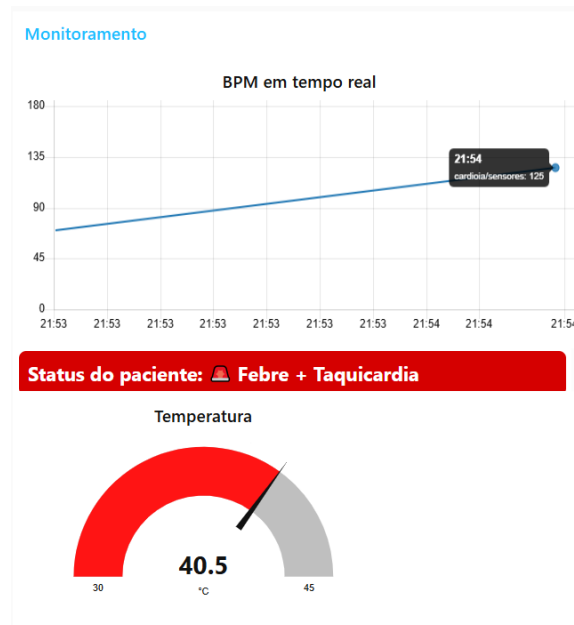
```
{
  "temperatura":28.8,"umidade":40,"bpm":70,"timestamp":10006,"cached":false,"hipotermia":true,"febre":false,"alerta_bpm":false,"bradicardia":false,"sinal_ausente":false,"status_pacien
  ermia"}
Conectando MQTT... OK
MQTT +
{"temperatura":36.3,"umidade":40,"bpm":70,"timestamp":20030,"cached":false,"hipotermia":false,"febre":false,"alerta_bpm":false,"bradicardia":false,"sinal_ausente":false,"status_pacien
  ermia"}

=== WIFI SIMULADO: OFFLINE ===
OFFLINE -> salvando no cache: 1/50
OFFLINE -> salvando no cache: 2/50
OFFLINE -> salvando no cache: 3/50
Clique detectado | Total na janela: 1

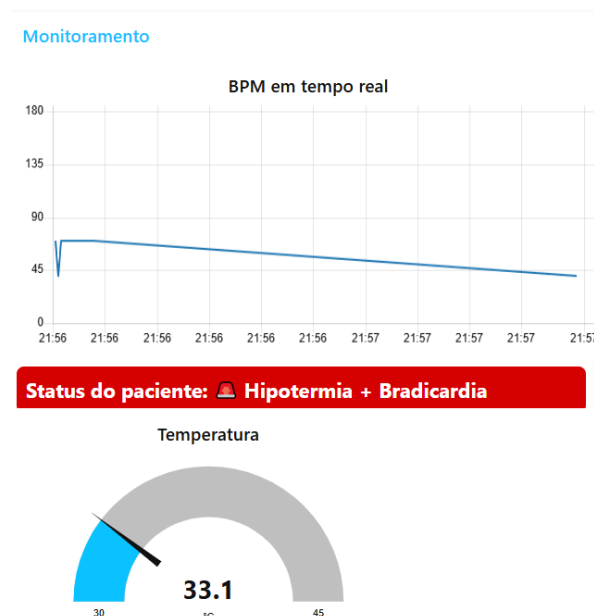
=== WIFI SIMULADO: ONLINE ===
Conectando MQTT... OK
Sincronizando cache: 3
MQTT +
{"temperatura":36.3,"umidade":40,"bpm":70,"timestamp":30053,"cached":true,"hipotermia":false,"febre":false,"alerta_bpm":false,"bradicardia":false,"sinal_ausente":false,"status_pacien
  ermia"}
MQTT +
{"temperatura":36.3,"umidade":40,"bpm":70,"timestamp":40060,"cached":true,"hipotermia":false,"febre":false,"alerta_bpm":false,"bradicardia":false,"sinal_ausente":false,"status_pacien
  ermia"}
MQTT +
{"temperatura":36.3,"umidade":40,"bpm":70,"timestamp":50067,"cached":true,"hipotermia":false,"febre":false,"alerta_bpm":false,"bradicardia":false,"sinal_ausente":false,"status_pacien
  ermia"}
Cache enviado com sucesso
MQTT +
{"temperatura":36.3,"umidade":40,"bpm":0,"timestamp":61544,"cached":false,"hipotermia":false,"febre":false,"alerta_bpm":false,"bradicardia":false,"sinal_ausente":true,"status_pacien
  ermia"}
MQTT +
```

**Figura 3 – Demonstração da resiliência offline e sincronização automática via MQTT.**

Além disso, o sistema foi capaz de detectar automaticamente diferentes estados fisiológicos simulados, exibindo alertas visuais em tempo real como mostram as Figuras 4 e 5.



**Figura 4 – Dashboard detectando febre e taquicardia.**



**Figura 5 – Dashboard detectando hipotermia e bradicardia.**

Os testes demonstraram:

- Comunicação MQTT funcionando corretamente;
- Dashboard atualizado em tempo real;

- Detecção automática de alertas clínicos;
- Resiliência offline;
- Sincronização automática após reconexão.

## 9 LIMITAÇÕES DA SOLUÇÃO

Por se tratar de uma simulação, os dados fisiológicos não representam medições médicas reais.

Além disso:

- O armazenamento em RAM não garante persistência após reinicialização do ESP32;
- O sistema utiliza sensores simulados;
- O broker MQTT utilizado é público;
- O dashboard roda localmente via Node-RED.

## 10 TRABALHOS FUTUROS

Como evolução futura, pretende-se:

- Integrar sensores biomédicos reais;
- Utilizar SPIFFS para persistência dos dados;
- Hospedar o dashboard em nuvem;
- Implementar banco de dados;
- Integrar modelos de Inteligência Artificial para previsão de risco cardíaco;
- Criar notificações automáticas para profissionais da saúde.

## 11 CONCLUSÃO

O desenvolvimento desta etapa permitiu compreender de forma prática como soluções de Edge Computing podem ser aplicadas na área da saúde.

Mesmo utilizando um ambiente de simulação, o projeto demonstrou:

- Coleta de sinais vitais;
- Processamento local;
- Resiliência offline;
- Sincronização automática;
- Comunicação em nuvem;
- Monitoramento em tempo real.

A solução desenvolvida representa uma arquitetura moderna e compatível com aplicações reais de monitoramento remoto de pacientes, especialmente em cenários onde a conectividade pode ser instável.

## 12 REFERÊNCIAS

WOKWI. *Wokwi Simulator*. Disponível em: <https://wokwi.com>. Acesso em: 12 maio 2026.

NODE-RED. *Flow-based programming for the Internet of Things*. Disponível em: <https://nodered.org>. Acesso em: 12 maio 2026.

EMQX. *EMQX MQTT Broker Platform*. Disponível em: <https://www.emqx.com>. Acesso em: 12 maio 2026.

IBM. *MQTT Protocol Overview*. Disponível em: <https://www.ibm.com/docs/en/ibm-mq/9.3?topic=overview-mqtt>. Acesso em: 12 maio 2026.

ARDUINO. *Arduino Documentation*. Disponível em: <https://docs.arduino.cc>. Acesso em: 12 maio 2026.

ESPRESSIF SYSTEMS. *ESP32 Documentation*. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/>. Acesso em: 12 maio 2026.

ADAFRUIT. *DHT22 Sensor Documentation*. Disponível em: <https://learn.adafruit.com/dht>. Acesso em: 12 maio 2026.

HIVEMQ. *MQTT Essentials*. Disponível em: <https://www.hivemq.com/mqtt-essentials/>. Acesso em: 12 maio 2026.

MICROSOFT AZURE. *What is Edge Computing?*. Disponível em: <https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-edge-computing/>. Acesso em: 12 maio 2026.

AMAZON WEB SERVICES (AWS). *What is the Internet of Things (IoT)?*. Disponível em: <https://aws.amazon.com/iot/what-is-the-internet-of-things/>. Acesso em: 12 maio 2026